Custom Image Classification Dataset and Model Development for Five Animal Classes

Shahriar Swanon (2231540642), Md. Abu Sufian Protik (2312234042), Khandakar Atikur Ahsan (2222120642), Md. Faiaz Bin Hayder (2222629042)

CSE445 – Machine Learning
Section: 6
Project Group No. 1

Abstract:

This project is focusing on developing a custom image classification dataset consisting of five animal categories like dog, cow, cat, lamb, and zebra with approximately 100+ image. Per class collected from online sources. The primary objective is to train a model which capable of achieving around 90% or more than 90% accuracy in classifying these images. In the initial phase, deep learning techniques were intentionally avoided aligning with course requirements. Instead, multiple traditional models, preprocessing strategies, and feature extraction approaches were explored to establish baseline performance. However, due to the limited size of the dataset, achieving strong feature representation and high accuracy using only classical methods proved challenging. But at the end of the project, we successfully got the targeted accuracy where three models crossed the 90% accuracy which was our final target in this project.

Dataset Overview:

Classes: Dog, Cow, Cat, Lamb, Zebra

Sources: Internet

Preprocessing: Converts to RGB, Saves as JPEG, Deletes corrupted files, Converts image size into 64x64 or 128x128

Methodology and Experiments:

A. Support Vector Machine (SVM) Model - Shahriar Swanon

In this project, a Support Vector Machine (SVM) model was implemented for image classification on a custom dataset of five animal classes: cat, dog, cow, lamb, and zebra. The dataset contained approximately 500+ images collected from online sources, making it relatively small and prone to inconsistencies. To ensure data quality, a cleaning step was performed using the PIL library, where each image was opened, converted into RGB format, saved as a valid JPEG file, and corrupted or unreadable images were removed from the dataset. After cleaning, the dataset was verified to confirm correct class distribution and proper structure. For preprocessing, all images were resized to 64×64 pixels and converted to grayscale to reduce computational complexity. Histogram of Oriented Gradients (HOG) feature extraction was applied to capture important edge and shape information, resulting in a more informative feature representation compared to raw pixel values. The dataset was split into training and testing sets using 80% training and 20% for testing to maintain class balance. Feature scaling was applied using Standard-Scaler to normalize the data, which is essential for SVM as it relies on distance-based calculations. The model was trained using an SVM with an RBF kernel, and hyperparameters were optimized using GridSearchCV to achieve better performance. The model was evaluated using accuracy, classification report, and confusion matrix. The results showed a training accuracy of 100% and a testing accuracy of approximately 59%, indicating that the model learned the training data very well but faced challenges in generalizing unseen data. This limitation is mainly due to the relatively small dataset size and variations in image background, lighting, and object appearance. Despite these challenges, the model demonstrates the capability of SVM combined with HOG features to perform multi-class image classification and provides a reasonable baseline for classical machine learning approaches on small-scale datasets.

B. Logistics Regression Model – Md. Faiaz Bin Hayder

In this project, we worked on classifying animal images into five categories like cat, dog, cow, lamb, and zebra using the Logistic Regression. The images were collected from online sources, and the original dataset was relatively small with around 500+ images, which is one of the main challenges we faced throughout the project. To deal with this limitation, we applied several data augmentation techniques using OpenCV, such as rotating images at different angles (±15° and ±30°), flipping them horizontally and vertically, adjusting brightness levels, and applying Gaussian blur. This helped us expand the dataset by roughly 10 times, giving the model more variety to learn from.

Before training, all images were resized into 128×128 pixels and converted in RGB. Pixel values were also normalized to fall within the (0, 1) range so that the model receives consistent input during training. One of the more important things in this project was the feature extraction. Since Logistic Regression cannot directly learn from raw image pixels the way deep learning models can, we had to carefully design a feature vector that captures meaningful information from each image. For this, we combined several types of features color statistics from RGB, HSV, and LAB color spaces to capture the color distribution of each animal, texture features using Local Binary Patterns (LBP) at multiple scales and Gabor filters to detect patterns like fur texture and zebra stripes, edge and gradient information using Canny edge detection and Sobel operators, shape descriptors using Hu Moments, 16-bin color histograms for each channel, and regional statistics by dividing the image into four quadrants. Together, these features gave us a detailed and rich representation of each image that Logistic Regression could work with effectively. For splitting the data, we used a stratified approach to maintain equal class distribution across training set (60%), validation set (20%), and testing set (20%). Feature scaling was applied using Standard-Scaler or, which is important for Logistic Regression since it is sensitive to differences in feature magnitudes. We also applied feature selection techniques including Select-KBest using ANOVA F-scores, Recursive Feature Elimination (RFE), and polynomial feature interactions to reduce dimensionality and keep only the most useful features. To find the best model, we tested a wide range of hyperparameter combinations covering different regularization strengths (C, 0.01 to 100), L1 and L2 penalties, different solvers like liblinear and lbfgs, and class weighting options all evaluated through 3-fold Stratified Cross-

Validation. We also applied probability calibration using Isotonic Regression and Sigmoid methods to further improve the model's prediction confidence. The final model was evaluated on the test set and achieved an overall accuracy of 82.8% which couldn't meet our target. Looking at each class results, the cat class performed the best with an accuracy of 98.6% (highest), which suggests that cats have more distinct visual features that the model could pick up on. On the other hand, cow and dog were the hardest classes to classify correctly, with accuracies of 75.7% and 76.7% respectively, while lamb and zebra reached 80.8% and 81.5%. The confusion matrix showed that most of the misclassifications happened between cow, dog, and lamb, which makes sense as these animals can look visually similar in terms of body shape and coat color, especially in varied backgrounds and lighting conditions. Overall, the project showed that with careful feature engineering, data augmentation, and proper hyperparameter tuning, Logistic Regression can still produce reasonably good results on image classification tasks without using deep learning, and it serves as a solid classical machine learning baseline for this type of problem.

C. K-Nearest Neighbor (KNN) Model - Md. Abu Sufian Protik

In this project, a K-Nearest Neighbors (KNN) model was implemented to perform image classification on a custom dataset consisting of five animal classes: cat, dog, cow, lamb, and zebra. The dataset contained approximately 500+ images collected from online sources, making it relatively small and subject to variations in lighting, orientation, and background noise. Initially, raw pixel values (64×64×3) were used as input features, resulting in a very high-dimensional feature space (12,288 features per image). This approach produced poor performance, achieving only around 35% test accuracy. The low accuracy was mainly due to the curse of dimensionality, where distance-based algorithms like KNN become ineffective in high-dimensional spaces, and raw pixel values fail to capture meaningful patterns.

To improve performance, several feature engineering and preprocessing techniques were applied. Instead of raw pixels, color histogram features were extracted using 32 bins for each RGB channel, reducing the feature size to 96 while preserving important color distribution information. Data

augmentation techniques, including horizontal flipping and rotation, were applied to increase the dataset size and improve robustness to variations in orientation. Feature standardization was performed to normalize the data, ensuring that all features contributed equally during distance calculation. Additionally, Principal Component Analysis (PCA) was implemented to further reduce dimensionality to approximately 45 components while retaining most of the variance, helping to remove noise and redundant information.

The dataset was then split into 80% training and 20% testing sets, and the KNN model was trained using Euclidean distance as the similarity metric. The optimal value of K was selected through experimentation, with K=3 providing the best performance. After applying all improvements, the model achieved test accuracy of approximately 93.67%, significantly exceeding the project requirement of 90%. The confusion matrix and classification metrics showed balanced performance across all classes, with only minor misclassifications, particularly between visually similar animals such as lamb and cow.

Despite its simplicity, the KNN model demonstrated strong performance when combined with effective feature engineering and dimensionality reduction techniques. However, it also has limitations, including higher computational cost during prediction due to distance calculations with all training samples. Overall, this approach highlights that with proper preprocessing, feature extraction, and dimensionality reduction, a classical machine learning algorithm like KNN can achieve high accuracy on image classification tasks without relying on pre-trained deep learning models.

D. Random Forest – Khandakar Atikur Ahsan

In this project, a Random Forest based image classification model was developed to classify a custom dataset consisting of five animal classes: cat, cow, dog, lamb, and zebra. The dataset was collected from a internet and refined to include exactly 100 images per class, resulting in a total of 500 images used for experimentation. All images were resized to 224×224 pixels to match the input requirements of the feature extraction model. The dataset was then split into training and testing sets using an 80:20 ratio, with 400 images used for training and 100 images for testing. To overcome the limitations of traditional feature extraction, a deep learning approach was incorporated by using the ResNet50 model pre-trained on ImageNet as a feature extractor. The top classification layers of ResNet50 were removed, and global average

pooling was applied to obtain high-level feature representations, resulting in 2,048 features per image. These features were then normalized using Standard-Scaler to ensure consistent input distribution for the classifier.

For the classification task, a Random Forest classifier was employed due to its ability to handle high-dimensional data and reduce overfitting through ensemble learning. The model was trained using 500 decision trees with optimized parameters, including square-root feature selection and parallel processing to improve efficiency. After training, the model was evaluated using the test dataset, where performance metrics such as accuracy, classification report, and confusion matrix were analyzed. The results demonstrated a high overall accuracy of approximately 98%, significantly exceeding the project requirement of 90%. The confusion matrix revealed strong performance across all classes, with perfect classification achieved for cat, lamb, and zebra, while minor misclassifications occurred in cow and dog classes due to visual similarities. In total, only 2 out of 100 test images were misclassified, indicating excellent generalization capability.

Additionally, the model was tested using real sample images, where predictions were displayed alongside confidence scores, further validating its robustness. All-important artifacts, including the trained model, scaler, and extracted features, were saved for future use. Overall, this project demonstrates that combining deep feature extraction using ResNet50 with a classical machine learning model like Random Forest can produce highly accurate results in image classification tasks, even with a relatively small custom dataset. This approach effectively leverages the strengths of both deep learning and ensemble learning techniques to achieve superior performance.

Result Comparison

Model	Accuracy
SVM	59%
Logistic Regression	82.8%
KNN	93.67%
Random Forest	98%

Conclusion

This project explored multiple machine learning approaches for classifying a custom animal image dataset consisting of five classes. Initially, traditional models such as Support Vector Machine and Logistic Regression were implemented to establish baseline performance, where SVM achieved 59% accuracy and Logistic Regression improved the results to 82.8% but it's not met our target which is 90%. Further improvements were observed with the K-Nearest Neighbors (KNN) model, which achieved 93.67% accuracy by addressing high-dimensional data issues using color histograms and PCA. Here our target was fulfilled but we want to more accurately, that's why we used deep feature method, though this was not our course thing. However, the best performance was obtained using the Random Forest model combined with deep feature extraction from ResNet50, achieving an overall accuracy of 98%, significantly surpassing the project target of 90%.

The results clearly demonstrate that model performance in image classification heavily depends on feature representation and data quality. Classical machine learning models can achieve strong performance when supported by effective feature engineering but integrating deep learning-based feature extraction provides a significant advantage in capturing complex visual patterns. Overall, this project highlights the importance of experimenting with multiple approaches, understanding model limitations, and selecting appropriate techniques based on the problem. The combination of deep feature extraction and ensemble learning proved to be the most effective solution for this task.